DSA

Data Structures and Algorithms



Complete Notes

Total Pages : 20



Build Stronger Problem Solving Skills

Topics Covered

1. Overview and Importance of DSA
   (What is DSA, Why to learn, Applications)
2. Arrays
   (Basics, Operations, Problems)
3. Strings
   (Basics, Common Problems)
4. Linked List
   (Singly, Doubly, Circular, Problems)
5. Stack
   (Basics, Applications, Problems)
6. Queue
   (Basics, Types, Problems)
7. Hashing
   (HashMap, HashSet, Problems)
8. Trees
   (Binary Tree, BST, Traversals, Problems)
9. Heaps / Priority Queue
   (Concept, Implementation, Problems)
10. Graphs
    (BFS, DFS, Shortest Path, Problems)
11. Recursion
    (Concept, Backtracking, Problems)
12. Searching Algorithms
    (Linear Search, Binary Search)
13. Sorting Algorithms
    (Bubble, Selection, Insertion, Merge, Quick, Heap)
14. Greedy Algorithms
    (Concept, Examples, Problems)
15. Dynamic Programming
    (Memoization, Tabulation, Common Problems)
16. Sliding Window & Two Pointers
    (Concept, Pattern, Problems)
17. Bit Manipulation
    (Basics, Operations, Problems)
18. Trie
    (Concept, Implementation, Problems)
19. Disjoint Set (Union-Find)
    (Concept, Applications, Problems)
20. System Design / Advanced DSA
    (Design Concepts, Real-world Questions)



What You Will Learn?

- Step by step explanation of each topic
- Important concepts and patterns
- Handwritten diagrams and dry run examples
- Java code for common problems
- Time and space complexity analysis
- Interview focused questions
- Tips and best practices

Goal



- Build strong problem solving skills
- Understand core DSA concepts
- Solve real interview questions
- Write optimized and clean code
- Be ready for product based company interviews

Rahul Yadav.

Overview of DSA Roadmap

1. **DSA Overview and Importance** (Page 1)
   - What is DSA ?
   - Why learn DSA ?
   - Applications and career scope
   - Topics to be covered (Roadmap)
2. **Arrays** (Page 2)
   - What is an array ?
   - Types, operations, time complexity
   - Important problems (Easy + Medium)
3. **Strings** (Page 3)
   - String operations
   - Important string algorithms
   - Common problems
4. **Linked List** (Page 4)
   - Singly, Doubly, Circular Linked List
   - Operations and implementations
   - Important problems
5. **Stack** (Page 5)
   - Concept and working
   - Applications (Expression, Parentheses, etc.)
   - Important problems
6. **Queue** (Page 6)
   - Types of queues (Simple, Circular, Deque, Priority Queue)
   - Operations and implementation
   - Important problems
7. **Hashing** (Page 7)
   - Hash function and hash table
   - Handling collisions
   - Applications and problems
8. **Trees** (Page 8)
   - Binary Tree, BST
   - Traversals (Inorder, Preorder, Postorder)
   - Important problems
9. **Heaps / Priority Queue** (Page 9)
   - Min Heap, Max Heap
   - Heap operations
   - Important problems (Top K, Merge K sorted, etc.)
10. **Graphs** (Page 10)
    - Graph representation (Adjacency List/Matrix)
    - BFS, DFS
    - Shortest Path Algorithms (Dijkstra, Bellman-Ford)
    - Important problems
11. **Recursion** (Page 11)
    - What is recursion ?
    - Recursion vs Iteration
    - Backtracking (concept)
    - Important problems
12. **Searching Algorithms** (Page 12)
    - Linear Search
    - Binary Search
    - Applications and problems
13. **Sorting Algorithms** (Page 13)
    - Bubble, Selection, Insertion
    - Merge Sort, Quick Sort, Heap Sort
    - Time and space complexity
    - When to use which sort ?
14. **Greedy Algorithms** (Page 14)
    - What is greedy approach ?
    - Characteristics
    - Common problems (Activity Selection, Huffman Coding, etc.)
15. **Dynamic Programming** (Page 15)
    - What is DP ?
    - Memoization vs Tabulation
    - Steps to solve DP problems
    - Important DP problems
16. **Sliding Window & Two Pointers** (Page 16)
    - Concept and when to use
    - Fixed size and variable size window
    - Important problems
17. **Bit Manipulation** (Page 17)
    - Bitwise operators
    - Common tricks and concepts
    - Important problems
18. **Trie** (Page 18)
    - What is Trie ?
    - Implementation
    - Applications and problems (Dictionary, Auto-complete)
19. **Disjoint Set (Union-Find)** (Page 19)
    - Concept and operations (Find, Union)
    - Path compression, Union by rank
    - Important problems (Connected components, etc.)
20. **System Design / Advanced DSA** (Page 20)
    - Design concepts for big problems
    - Combine multiple data structures
    - Real-world and interview questions
    - Tips for practice and next steps

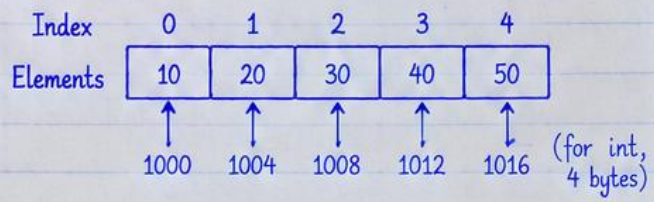
Rahul Yadav.

2. Arrays in Data Structures

1 What is an Array ?

- An array is a collection of elements of same data type stored in contiguous memory locations.
- Elements can be accessed using index (starting from 0).
- Provides random access.
- Size is fixed (in most languages like Java).

2 Array Representation in Memory



- Elements are stored in contiguous memory locations.
- Address of element = Base Address + (index × size)

3 Declaration and Initialization (Java)

```
// Declaration
int[] arr;
// Initialization
arr = new int[5];   // size 5
// Declaration + Initialization
int[] nums = {10, 20, 30, 40, 50};
```

4 Accessing Elements

```
int[] arr = {10, 20, 30, 40, 50};
System.out.println(arr[0]);   // 10
System.out.println(arr[3]);   // 40
```

- Access time complexity : O(1)

5 Basic Operations

- Traversal : Visit all elements
- Insertion : Add an element (usually at end)
- Deletion : Remove an element
- Searching : Find an element
- Updating : Modify an element

6 Time Complexity

Operation	Time Complexity
Access	O(1)
Traversal	O(n)
Search (Linear)	O(n)
Search (Binary) (sorted array)	O(log n)
Insertion (at end)	O(1)
Insertion (at beginning/middle)	O(n)
Deletion (at end)	O(1)
Deletion (at beginning/middle)	O(n)

7 Traversal Example

```
int[] arr = {10, 20, 30, 40, 50};
for (int i = 0; i < arr.length; i++) {
    System.out.print(arr[i] + " ");
}
// Output: 10 20 30 40 50
```

8 Insertion Example (at a specific position)

```
int[] arr = {10, 20, 40, 50};
int pos = 2;  // insert 30 at index 2
int val = 30;
for (int i = arr.length - 1; i > pos; i--) {
    arr[i] = arr[i - 1];
}
arr[pos] = val;
// Result: 10 20 30 40 50
```

9 Deletion Example (at a specific position)

```
int[] arr = {10, 20, 30, 40, 50};
int pos = 2;  // delete element at index 2 (30)
for (int i = pos; i < arr.length - 1; i++) {
    arr[i] = arr[i + 1];
}
// Last element is left duplicate, logical size reduces
// Result: 10 20 40 50
```

10 Searching Example (Linear Search)

```
int[] arr = {10, 20, 30, 40, 50};
int key = 40;
int index = -1;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == key) {
        index = i;
        break;
    }
}
System.out.println(index);  // Output: 3
```

11 Searching Example (Binary Search)

```
int[] arr = {10, 20, 30, 40, 50};  // sorted array
int key = 40;
int left = 0, right = arr.length - 1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (arr[mid] == key) {
        System.out.println(mid);  // Output: 3
        break;
    } else if (arr[mid] < key) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}
```

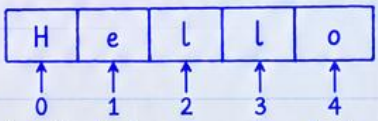
Rahul Yadav.

4. Strings in Data Structures

1 What is a String ?

- A string is a sequence of characters.
- In Java, String is a class (not a primitive type).
- Strings are immutable (cannot be changed after creation).
- Used to store and manipulate text data.
- Commonly used in real-world applications like search, parsing, validation, etc.

2 Representation in Memory



Each character is stored using Unicode (2 bytes) in Java (UTF-16).

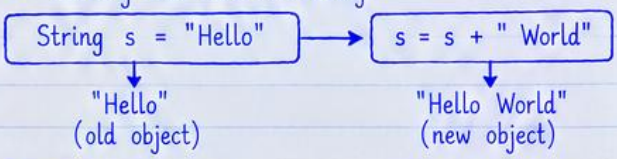
- Length of string = number of characters.
- Indexing starts from 0 to n - 1.
- Access character : s.charAt(index)
- Time Complexity : O(1)

3 String Declaration and Initialization (Java)

```
// Different ways to create a string
String s1 = "Hello";
String s2 = new String("World");
char[] ch = {'J', 'a', 'v', 'a'};
String s3 = new String(ch);
String s4 = "Rahul Yadav";
```

4 String is Immutable

- Once a string is created, it cannot be changed.
- Any modification creates a new string object in memory.
- This makes strings safe and helps in security, caching and multi-threading.



5 Common String Operations in Java

Operation	Method	Example	Time Complexity
Length	s.length()	s.length()	O(1)
Character at index	s.charAt(i)	s.charAt(0)	O(1)
Substring	s.substring(i,j)	s.substring(1, 4)	O(n)
Concatenation	s1 + s2	s1 + " " + s2	O(n)
Compare	s.equals()	s1.equals(s2)	O(n)
Lexicographical	s.compareTo()	s1.compareTo(s2)	O(n)
Find character	s.indexOf()	s.indexOf('a')	O(n)
Replace	s.replace()	s.replace('a','b')	O(n)
Split	s.split()	s.split(",")	O(n)
Trim	s.trim()	s.trim()	O(n)

6 String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread Safe	Yes	No	Yes
Performance	Slower	Faster	Slower (than SB)
Use Case	Fixed text	Frequent modification (single thread)	Multi-threaded environment

7 Example: StringBuilder

```
StringBuilder sb = new StringBuilder();
sb.append("Hello");
sb.append(" ");
sb.append("World");
System.out.println(sb.toString());   // Hello World
```

8 Common Problems and Examples

8.1 Reverse a String

```
String s = "Hello";
StringBuilder sb = new StringBuilder();
for (int i = s.length() - 1; i >= 0; i--) {
    sb.append(s.charAt(i));
}
System.out.println(sb.toString());   // olleH
```

8.2 Check Palindrome

```
String s = "madam";
String rev = new StringBuilder(s)
    .reverse().toString();
if (s.equals(rev)) {
    System.out.println("Palindrome");
} else {
    System.out.println("Not Palindrome");
}
```

8.3 Check Anagram

```
String s1 = "listen";
String s2 = "silent";
if (s1.length() != s2.length()) {
    System.out.println("Not Anagram");
} else {
    char[] a = s1.toCharArray();
    char[] b = s2.toCharArray();
    Arrays.sort(a);
    Arrays.sort(b);
    if (Arrays.equals(a, b))
        System.out.println("Anagram");
    else
        System.out.println("Not Anagram");
}
```

8.4 Count Characters

```
String s = "hello world";
int count = 0;
for (int i = 0; i < s.length(); i++) {
    if (s.charAt(i) != ' ') {
        count++;
    }
}
System.out.println("Character count: " + count);
// Output: 10
```

8.5 First Non-Repeating Character

```
String s = "aabbcd";
int[] freq = new int[256];
for (char c : s.toCharArray()) {
    freq[c]++;
}
for (char c : s.toCharArray()) {
    if (freq[c] == 1) {
        System.out.println("First non-repeating: " + c);
        break;
    }
} // Output: c
```

9 Time and Space Complexity

- Access character : O(1)
- Length : O(1)
- Concatenation : O(n)
- Substring : O(n)
- Search (indexOf) : O(n)
- Replace : O(n)
- Split : O(n)
- Extra space : O(n) (for new string)

10 Applications of Strings

- Text processing and parsing
- Search and autocomplete
- Validation (emails, phone numbers, etc.)
- Data compression
- File handling
- Web development
- Natural Language Processing (NLP)
- Competitive programming

Rahul Yadav.

5. Linked Lists in Data Structures

1 What is a Linked List ?

- A linked list is a linear data structure in which elements (nodes) are stored in non-contiguous memory locations.
- Each node contains two parts:
  - Data (value)
  - Reference (address of next node)
- Nodes are linked using pointers (references).
- Size is dynamic (can grow or shrink at runtime).
- More flexible than arrays (no fixed size).

2 Node Structure (Java)

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

3 Types of Linked Lists

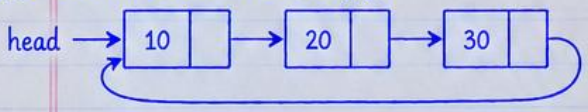
(a) Singly Linked List



(b) Doubly Linked List



(c) Circular Linked List (Singly)



4 Basic Operations

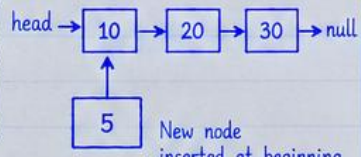
- Traversal : Visit all nodes
- Insertion : Add a node (beginning, end, middle)
- Deletion : Remove a node (beginning, end, middle)
- Search : Find a node with given value
- Update : Modify node value
- Length : Count total number of nodes

5 Time Complexity

Operation	Singly LL	Doubly LL	Circular LL
Access	O(n)	O(n)	O(n)
Search	O(n)	O(n)	O(n)
Insertion (at beginning)	O(1)	O(1)	O(1)
Insertion (at end)	O(n)	O(1)	O(1)
Deletion (at beginning)	O(1)	O(1)	O(1)
Deletion (at end)	O(n)	O(1)	O(n)

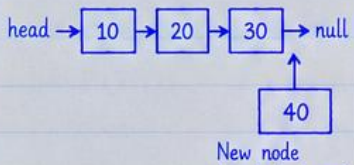
6 Insertion at Beginning (Java)

```
public void insertAtBeginning(int val) {
    Node newNode = new Node(val);
    newNode.next = head;
    head = newNode;
}
```



7 Insertion at End (Java)

```
public void insertAtEnd(int val) {
    Node newNode = new Node(val);
    if (head == null) {
        head = newNode;
        return;
    }
    Node temp = head;
    while (temp.next != null) {
        temp = temp.next;
    }
    temp.next = newNode;
}
```



8 Deletion of a Node (Java)

```
public void deleteNode(int key) {
    if (head == null) return;

    if (head.data == key) {
        head = head.next;
        return;
    }

    Node temp = head;
    while (temp.next != null && temp.next.data != key){
        temp = temp.next;
    }

    if (temp.next != null) {
        temp.next = temp.next.next;
    }
}
```

9 Traversal (Java)

```
public void traverse() {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    }
    System.out.println("null");
}
```

10 Advantages and Disadvantages

Advantages

- Dynamic size
- Efficient insertion and deletion
- No need for contiguous memory
- Easy to implement stacks, queues, and other data structures

Disadvantages

- Random access is not allowed
- Extra memory for pointers
- Slower access compared to arrays
- More complex to implement

Rahul Yadav.

6. Stacks in Data Structures

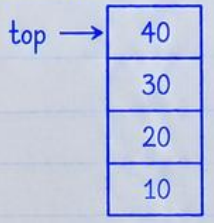
1 What is a Stack ?

- A stack is a linear data structure that follows LIFO (Last In First Out) principle.
- Insertion and deletion are performed only at one end called the top.
- The last element inserted is the first one to be removed.
- Commonly used in function calls, expression evaluation, backtracking, etc.

2 Stack Operations

- push(x) : Insert an element at the top
- pop() : Remove and return the top element
- peek() : Return the top element without removing
- isEmpty() : Check if stack is empty
- size() : Return number of elements in stack

3 Stack Representation



Last In First Out (LIFO)

4 Stack Implementation using Array (Java)

```
class StackUsingArray {
    int[] arr;
    int top;
    int capacity;

    StackUsingArray(int size) {
        arr = new int[size];
        top = -1;
        capacity = size;
    }
    public void push(int x) {
        if (isFull()) {
            System.out.println("Stack Overflow");
            return;
        }
        arr[++top] = x;
    }
    public int pek() {
        if (isEmpty()) {
            System.out.println("Stack Underflow");
            return -1;
        }
        return arr[top--];
    }
    public int peek() {
        if (isEmpty()) return -1;
        return arr[top];
    }
    public boolean isEmpty() {
        return top == -1;
    }
    public boolean isFull() {
        return top == capacity - 1;
    }
    public int size() {
        return top + 1;
    }
}
```

5 Stack Implementation using Java Collection

```
import java.util.Stack;

public class StackExample {
  public static void main(String[] args) {
    Stack<Integer> stack = new Stack<>();
    stack.push(10);
    stack.push(20);
    stack.push(30);

    System.out.println("Top element: " + stack.peek());
    System.out.println("Pop: " + stack.pop());
    System.out.println("Size: " + stack.size());
    System.out.println("Is Empty: " + stack.isEmpty());
  }
}
```

6 Time Complexity

Operation	Time Complexity
push()	O(1)
pop()	O(1)
peek()	O(1)
isEmpty()	O(1)
size()	O(1)

7 Applications of Stack

- Function call stack (recursion)
- Expression conversion (Infix to Postfix/Prefix)
- Expression evaluation (Postfix/Prefix)
- Backtracking (e.g., maze, N-Queens)
- Undo/Redo operations in editors
- Browser history
- Syntax parsing (e.g., balanced parentheses)

8 Common Problems

1. Check for balanced parentheses.
2. Infix to Postfix conversion.
3. Evaluate postfix expression.
4. Next greater element.
5. Largest rectangle in histogram.
6. Stock span problem.
7. Implement stack using queue.
8. Implement queue using stack.

Rahul Yadav.

7. Queues in Data Structures

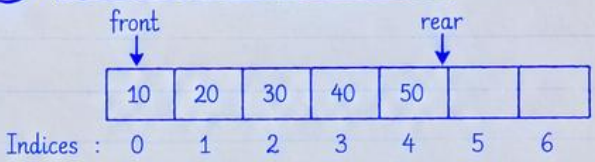
1 What is a Queue ?

- A queue is a linear data structure that follows FIFO (First In First Out) principle.
- Insertion is done at the rear (enqueue) and deletion is done from the front (dequeue).
- Elements are processed in the same order as they are inserted.
- Commonly used in scheduling, buffering,
- BFS in graphs, etc.

2 Queue Operations

- enqueue(x) : Insert an element at the rear.
- dequeue() : Remove and return the front element.
- peek() : Return the front element without removing,
- isEmpty() : Check if queue is empty.
- size() : Return number of elements.

3 Basic Queue Representation



4 Queue Implementation using Array (Java)

```
class QueueUsingArray {
    int[] arr;
    int front, rear, size;

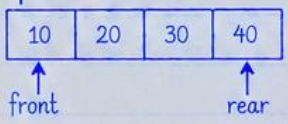
    QueueUsingArray(int n) {
        arr = new int[n];
        front = -1;
        rear = -1;
        size = n;
    }
    public boolean isEmpty() {
        return front == -1;
    }

    public boolean isFull() {
        return rear == size - 1;
    }
    public void enqueue(int x) {
        if (isFull()) {
            System.out.println("Queue Overflow");
            return;
        }
        if (front == -1) front = 0;
        arr[++rear] = x;
    }

    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue Underflow");
            return -1;
        }
        int val = arr[front++];
        if (front > rear) {
            front = rear = -1;  // queue becomes empty
        }
        return val;
    }
    public int peek() {
        if (isEmpty()) return -1;
        return arr[front];
} }
```

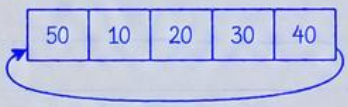
5 Types of Queues

(a) Simple Queue



Standard FIFO queue.

(b) Circular Queue



Last index connects to first index. Useful to utilize memory efficiently.

(c) Deque (Double Ended Queue)



Insertion and deletion can be performed from both ends.

(d) Priority Queue



Elements are removed based on priority (e.g., smallest or largest).

6 Queue Implementation using Java Collection

```
import java.util.*;

public class QueueExample {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();
        q.offer(10);
        q.offer(20);
        q.offer(30);
        System.out.println("Front: " + q.peek());
        System.out.println("Dequeue: " + q.poll());
        System.out.println("Size: " + q.size());
    }
}
```

7 Time Complexity

Operation	Array Implementation	LinkedList Implementation
Enqueue	O(1)	O(1)
Dequeue	O(1)	O(1)
Peek	O(1)	O(1)
isEmpty	O(1)	O(1)
size	O(1)	O(1)

8 Applications of Queue

- CPU scheduling (Round Robin)
- Print spooling
- BFS in graphs
- Handling requests (Web servers)
- Cache management
- Messaging systems (e.g., RabbitMQ, Kafka)
- Real-time data buffering (audio/video)

9 Common Problems

1. Implement queue using array.
2. Implement circular queue.
3. Implement queue using linked list.
4. Reverse a queue.
5. Find the first non-repeating element in a stream.
6. Implement a queue using two stacks.
7. Design a priority queue.

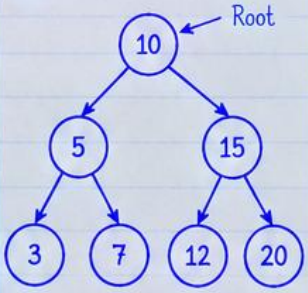
Rahul Yadav.

8. Trees in Data Structures

1 What is a Tree ?

- A tree is a hierarchical non-linear data structure consisting of nodes connected by edges.
- The topmost node is called the root.
- Each node can have zero or more child nodes (subtrees).
- A tree does not contain cycles.
- Trees are widely used in searching, sorting, databases, file systems, etc.

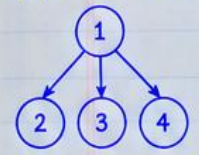
2 Terminology



- **Node** : An element of tree
- **Root** : Topmost node
- **Parent** : Immediate predecessor
- **Child** : Immediate successor
- **Leaf Node** : Node with no children
- **Internal Node** : Node with at least one child
- **Subtree** : A tree formed by a node and its descendants
- **Height** : Longest path from node to leaf
- **Depth** : Number of edges from root to a node

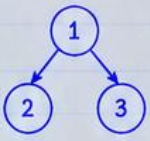
3 Types of Trees

(a) General Tree



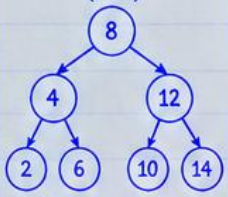
Each node can have any number of children.

(b) Binary Tree



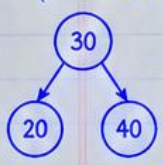
Each node has at most 2 children (left and right).

(c) Binary Search Tree (BST)

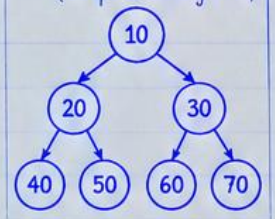


Left < Root < Right

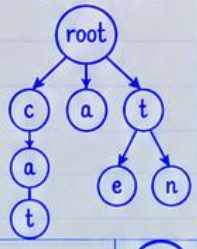
(d) AVL Tree (Balanced BST)



(e) Heap Tree (Complete Binary Tree)



(f) Trie (Prefix Tree)



4 Binary Tree Node Structure (Java)

```
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    TreeNode(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}
```

5 Tree Traversals

- **Preorder** (Root, Left, Right) : 10 5 3 7 15 12 20
- **Inorder** (Left, Root, Right) : 3 5 7 10 12 15 20
- **Postorder** (Left, Right, Root) : 3 7 5 12 20 15 10
- **Level Order** (BFS) : 10 5 15 3 7 12 20

6 Inorder Traversal (Java)

```
public void inorder(TreeNode root) {
    if (root == null) return;
    inorder(root.left);
    System.out.print(root.data + " ");
    inorder(root.right);
}
```

7 Preorder and Postorder Traversal (Java)

Preorder Traversal

```
public void preorder(TreeNode root) {
    if (root == null) return;
    System.out.print(root.data + " ");
    preorder(root.left);
    preorder(root.right);
}
```

Postorder Traversal

```
public void postorder(TreeNode root) {
    if (root == null) return;
    postorder(root.left);
    postorder(root.right);
    System.out.print(root.data + " ");
}
```

8 Level Order Traversal (Java)

```
public void levelOrder(TreeNode root) {
    if (root == null) return;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        TreeNode node = q.poll();
        System.out.print(node.data + " ");
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
}
```

9 Time Complexity

Operation	Time Complexity
Insert (BST)	O(h) (O(log n) for balanced)
Search (BST)	O(h) (O(log n) for balanced)
Delete (BST)	O(h)
Inorder Traversal	O(n)
Preorder Traversal	O(n)
Postorder Traversal	O(n)
Level Order Traversal	O(n)
Space Complexity (Recursion)	O(h)

10 Applications of Trees

- Hierarchical data representation (file systems)
- Database indexing (B-Trees, B+ Trees)
- Expression parsing (syntax trees)
- Searching and sorting (BST, AVL trees)
- Priority queues (Heap)
- Auto-complete (Trie)
- Decision making (AI, game trees)
- Compression (Huffman trees)
- Representing HTML/XML (DOM trees)

11 Common Interview Problems

1. Inorder, preorder and postorder traversal.
2. Level order traversal.
3. Maximum depth / height of a tree.
4. Check if a tree is balanced.
5. Lowest Common Ancestor (LCA).
6. Validate Binary Search Tree.
7. Serialize and deserialize a binary tree.

Rahul Yadav.

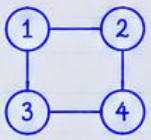
9. Graphs in Data Structures

1 What is a Graph ?

- A graph is a non-linear data structure consisting of a set of vertices (nodes) and a set of edges that connect pairs of vertices.
- Graphs can be used to represent real-world problems like social networks, maps, routing, dependency, etc.
- A graph can be directed or undirected, weighted or unweighted.
- It can contain cycles.
- Notation : G = (V, E) where V = vertices, E = edges.

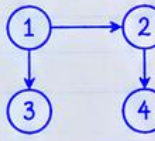
2 Types of Graphs

(a) Undirected Graph



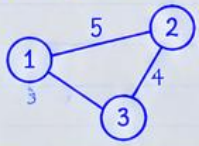
Edges have no direction.

(b) Directed Graph (Digraph)



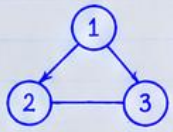
Edges have direction.

(c) Weighted Graph



Edges have weights.

(d) Unweighted Graph



All edges have same weight (assumed 1).

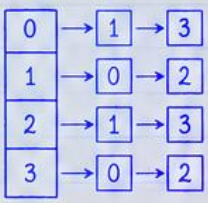
3 Graph Representation

(a) Adjacency Matrix

	0	1	2	3
0	0	1	0	1
1	1	0	1	0
2	0	1	0	1
3	1	0	1	0

- 2D array of size V × V
- matrix[i][j] = 1 if edge exists between i and j
- Space Complexity : $O(V^2)$
- Good for dense graphs

(b) Adjacency List



- Array/List of lists
- Each vertex stores a list of its adjacent vertices
- Space Complexity : O(V+E)
- Good for sparse graphs (most common)

4 Graph Implementation using Adjacency List (Java)

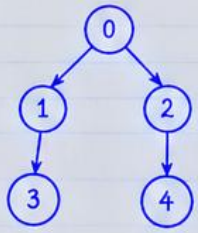
```
import java.util.*;
class Graph {
    private int V;
    private List<List<Integer>> adj;

    public Graph(int v) {
        V = v;
        adj = new ArrayList<>();
        for (int i = 0; i < v; i++) {
            adj.add(new ArrayList<>());
        }
    }
    public void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u);    // for undirected graph
    }
}
```

5 Graph Traversals

(a) Breadth First Search (BFS)

- Traverses level by level.
- Uses a Queue.
- Finds shortest path in unweighted graphs.

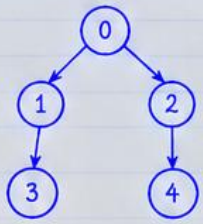


BFS (Java)

```
public void bfs(int start) {
    boolean[] visited = new boolean[V];
    Queue<Integer> q = new LinkedList<>();
    q.add(start);
    visited[start] = true;
    while (!q.isEmpty()) {
        int node = q.poll();
        System.out.print(node + " ");
        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.add(neighbor);
            }
        }
    }
}
```

(b) Depth First Search (DFS)

- Traverses as deep as possible before backtracking.
- Uses a Stack (or recursion).
- Useful for cycle detection, connected components, etc.



DFS (Java - Recursive)

```
public void dfs(int node, boolean[] visited) {
    visited[node] = true;
    System.out.print(node + " ");
    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor]) {
            dfs(neighbor, visited);
        }
    }
}

// Call DFS
boolean[] visited = new boolean[V];
dfs(0, visited);
```

6 Time Complexity

Operation	Adjacency Matrix	Adjacency List
Add Vertex	$O(V^2)$	O(1)
Add Edge	O(1)	O(1)
Remove Edge	O(1)	O(V)
Check Edge	O(1)	O(V)
BFS	$O(V^2)$	O(V+E)
DFS	$O(V^2)$	O(V+E)
Space	$O(V^2)$	O(V+E)

7 Common Applications

- Social networks (friends, followers)
- Maps and navigation (shortest path)
- Web page ranking (PageRank)
- Dependency resolution (build tools)
- Network routing (internet, telecom)
- Recommendation systems
- Scheduling and task dependencies
- Detecting cycles (deadlock detection)
- Connected components (grouping)

8 Common Interview Problems

1. Number of connected components.
2. Detect if a graph has a cycle (directed/undirected).
3. Find the shortest path (BFS / Dijkstra).
4. Clone a graph.
5. Find all paths from source to destination.
6. Topological sorting (for DAG).
7. Find strongly connected components.
8. Minimum spanning tree (Prim's / Kruskal's).
9. Word Ladder problem (BFS).
10. Number of islands (Grid as a graph).

Rahul Yadav.

10. Heaps and Priority Queues

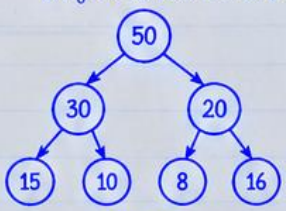
1 What is a Heap ?

- A heap is a complete binary tree that satisfies the heap property.
- In a Max Heap, the parent node is greater than or equal to its children.
- In a Min Heap, the parent node is smaller than or equal to its children.
- Heaps are commonly implemented using an array.
- Used to implement Priority Queue and in heap sort algorithm.
- Time complexity for insertion and deletion is $O(\log n)$.

2 Types of Heaps

(a) Max Heap

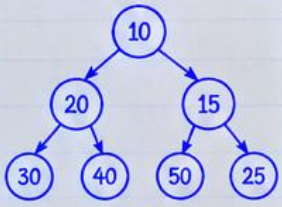
- Parent ≥ Children (largest element at root)



Max Heap (Complete Binary Tree)

(b) Min Heap

- Parent ≤ Children (smallest element at root)



Min Heap (Complete Binary Tree)

3 Array Representation of Heap

- Heaps are usually implemented using an array (like a complete binary tree).
- For a node at index i (0-based indexing):

```
Left child index  = 2 * i + 1
Right child index = 2 * i + 2
Parent index      = (i - 1) / 2
```

Example Array (Max Heap)

Index :	0	1	2	3	4	5	6
	50	30	20	15	10	8	16

4 Priority Queue

- A Priority Queue is a special queue where each element has a priority.
- Elements are served based on priority (highest or lowest first).
- It can be implemented using:
  - Heap (most common)
  - Balanced BST (e.g., TreeMap in Java)
- Java provides PriorityQueue class (by default it is a Min Heap).
- Time complexity for insert, delete, peek is $O(\log n)$.

5 Heap Operations (Min Heap)

- insert(x) : Add a new element and maintain heap property (heapify up).
- deleteMin() : Remove and return the root element.
- peek() : Return the root element.
- heapify(i) : Maintain heap property from index i.
- buildHeap() : Convert an array into a heap.
- size() : Return number of elements.

6 Java Implementation using PriorityQueue

```
import java.util.*;
public class HeapExample {
    public static void main(String[] args) {
        // Min Heap (default)
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        minHeap.offer(10);
        minHeap.offer(5);
        minHeap.offer(20);
        System.out.println("Min Heap (Priority Queue): " + minHeap);
        System.out.println("Peek: " + minHeap.peek());  // 5
        System.out.println("Poll: " + minHeap.poll());  // 5
    }   System.out.println("After Poll: " + minHeap);   // [10, 20]
}
```

7 Heapify (Min Heap) Algorithm

```
public void heapify(int[] arr, int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;
    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        int temp = arr[i];
        arr[i] = arr[smallest];
        arr[smallest] = temp;
        heapify(arr, n, smallest);
    }
}
```

8 Heap Sort (Ascending Order)

```
public void heapSort(int[] arr) {
    int n = arr.length;
    // Step 1: Build Min/Max Heap (using Max Heap for ascending order)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapifyMax(arr, n, i);

    // Step 2: Extract elements one by one
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapifyMax(arr, i, 0);
    }
}
```

Time Complexity (Heap Sort)

- Build Heap : $O(n)$
- Sorting : $O(n \log n)$
- Space : $O(1)$ (in-place sort)

9 Time Complexity of Heap Operations

Operation	Time Complexity
Insert (offer)	$O(\log n)$
Delete (poll)	$O(\log n)$
Peek	$O(1)$
Build Heap (from array)	$O(n)$
Heapify	$O(\log n)$
Heap Sort	$O(n \log n)$

10 Applications of Heaps

- Implementing Priority Queue
- CPU scheduling (process with highest priority)
- Dijkstra's shortest path algorithm
- Prim's algorithm (Minimum Spanning Tree)
- Finding Kth largest/smallest element
- Median in a data stream
- Top K frequent elements
- Merging K sorted lists
- Real-time systems and event handling

11 Common Interview Problems

1. Implement a Min Heap / Max Heap.
2. Implement Priority Queue using Heap.
3. Find Kth largest element in an array.
4. Find Kth smallest element in an array.
5. Sort a nearly sorted array.
6. Merge K sorted lists.
7. Find median from a data stream.
8. Find top K frequent elements.
9. Implement heap sort.
10. Convert an array to a heap.

Rahul Yadav.

11. Recursion and Backtracking

1 What is Recursion ?

- Recursion is a technique where a function calls itself to solve a smaller instance of the same problem.
- It breaks a big problem into smaller subproblems until it reaches a base case.
- Each recursive call is stored in the call stack.
- Commonly used in tree traversal, graph traversal, backtracking, divide and conquer, etc.

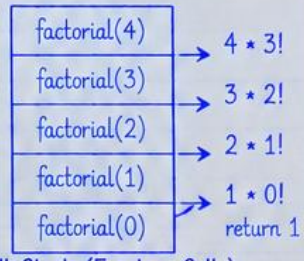
2 Components of Recursion

- Base Case : Condition to stop the recursion.
- Recursive Call : Function calls itself with a smaller input.
- Problem Size : Input is reduced in each call.
- Call Stack : Each call is stored in the stack memory.
- Return : The result is returned back from each call.

3 How Recursion Works ? (Call Stack)

Example: Factorial of n

```
public int factorial(int n) {
    if (n == 0) return 1;  // base case
    return n * factorial(n - 1); // recursive call
}
```



Call Stack (Function Calls)

4 Recursion vs Iteration

Recursion	Iteration
Function calls itself	Uses loops (for/while)
Uses call stack	No call stack overhead
More readable for complex problems	Better for simple problems
Higher memory usage	Lower memory usage
Risk of stack overflow	No stack overflow risk

5 Types of Recursion

- Direct Recursion : A function calls itself directly.
- Indirect Recursion : A function calls another function which calls the first function.
- Tail Recursion : Recursive call is the last statement.
- Head Recursion : Recursive call is the first statement.
- Tree Recursion : Function calls itself multiple times.

6 Backtracking

- Backtracking is a technique to solve problems by trying all possible solutions and abandoning those that do not satisfy the constraints.
- It builds the solution step by step and removes the choice (backtracks) if it leads to an invalid solution.
- Commonly used in problems like N-Queens, Sudoku, Rat in a Maze, Permutations, Combinations, etc.

7 Backtracking Example: N-Queens (Java)

```
public class NQueens {
    public void solve(int row, int n, char[][] board) {
        if (row == n) {
            printBoard(board);
            return;
        }
        for (int col = 0; col < n; col++) {
            if (isSafe(row, col, board)) {
                board[row][col] = 'Q';
                solve(row + 1, n, board); // backtrack
                board[row][col] = '.';    // remove choice
    }   }
    }
    private boolean isSafe(int row, int col, char[][] board) {
        // check column, left diagonal and right diagonal
        return true; // simplified
    }
}
```

N-Queens Example (n = 4)

Q	.	.	.
.	.	Q	.
.	Q	.	.
.	.	.	Q

8 Other Common Recursion Examples

1. Fibonacci Series
```
public int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```
2. Power of a Number
```
public int power(int x, int n) {
    if (n == 0) return 1;
    return x * power(x, n - 1);
}
```
3. Sum of First N Natural Numbers
```
public int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n - 1);
}
```

9 Time and Space Complexity

Operation	Time Complexity	Space Complexity
Recursion (simple)	O(n)	O(n) (call stack)
Backtracking	$O(2^n)$ (worst case)	O(n) (call stack)
N-Queens	O(n!)	O(n)
Fibonacci (naive)	$O(2^n)$	O(n)
Fibonacci (with memoization)	O(n)	O(n)

10 Applications

- Tree traversal (preorder, inorder, postorder)
- Graph traversal (DFS)
- Solving puzzles (N-Queens, Sudoku)
- Generating permutations and combinations
- Divide and Conquer algorithms (Merge Sort, Quick Sort)
- Path finding (Maze, Rat in a Maze)
- Expression evaluation (Parsing)
- AI and game solving (Chess, etc.)

11 Common Interview Problems

1. Fibonacci number (with and without memoization)
2. N-Queens problem
3. Sudoku solver
4. Rat in a Maze
5. Generate all permutations
6. Generate all combinations
7. Subsets / Subset sum
8. Word search in a matrix
9. Palindrome partitioning
10. Solve crossword puzzle

Rahul Yadav.

12. Searching Algorithms in Data Structures

1 What is Searching ?

- Searching is the process of finding the position of a target element in a data structure.
- It returns the index (position) of the element if found, otherwise -1.
- Searching is a fundamental operation used in many real-world applications like databases, search engines, file systems, etc.

2 Types of Searching Algorithms

- Linear Search (Sequential Search) : Works on both sorted and unsorted data.
- Binary Search : Works only on sorted data.
- Other Searching Techniques : Jump Search, Interpolation Search, Exponential Search, etc.

3 Linear Search (Sequential Search)

- Checks each element one by one until the element is found or the array ends.
- Can be used on both sorted and unsorted arrays.
- Time Complexity: O(n)
- Space Complexity: O(1)

Java Implementation (Linear Search)

```
public int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;  // element found
        }
    }
    return -1;  // element not found
}
```

4 Linear Search Example

Array:

10	5	8	20	15	30

Search target = 20

Time Complexity : O(n)
Space Complexity : O(1)

Step 1: 10 ≠ 20 → continue
Step 2: 5 ≠ 20 → continue
Step 3: 8 ≠ 20 → continue
Step 4: 20 = 20 → found at index 3

5 Binary Search

- Works only on sorted arrays.
- Repeatedly divides the search space into half.
- Compares the middle element with the target value.
- If target is smaller → search in left half.
- If target is larger → search in right half.
- Time Complexity: O(log n)
- Space Complexity: O(1) [Iterative]

6 Binary Search Example

Sorted Array:

2	4	6	8	10	12	14	16

Search target = 10

Step 1: mid = 3 (8) → 10 > 8 → search right half
Step 2: mid = 5 (12) → 10 < 12 → search left half
Step 3: mid = 4 (10) → 10 = 10 → **found at index 4**

7 Java Implementation (Binary Search - Iterative)

```
public int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

8 Java Implementation (Binary Search - Recursive)

```
public int binarySearchRecursive(int[] arr, int left, int right, int target) {
    if (left > right) return -1;

    int mid = left + (right - left) / 2;
    if (arr[mid] == target) {
        return mid;
    } else if (arr[mid] < target) {
        return binarySearchRecursive(arr, mid + 1, right, target);
    } else {
        return binarySearchRecursive(arr, left, mid - 1, target);
    }
}
```

9 Time and Space Complexity

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search (Iterative)	O(1)	O(log n)	O(log n)	O(1)
Binary Search (Recursive)	O(1)	O(log n)	O(log n)	O(log n) (call stack)

10 Other Searching Algorithms

- Jump Search : Works on sorted arrays, jump by fixed steps ($\sqrt{n}$).
- Interpolation Search : Improved version of binary search for uniformly distributed data.
- Exponential Search : For unbounded sorted arrays.
- Ternary Search : Divides array into three parts (used in optimization problems).

11 Applications of Searching

- Search engines (Google, Bing)
- Database search and indexing
- Finding records in large datasets
- File system search
- Dictionary word search (autocomplete)
- Searching in logs and console outputs
- IP address lookup
- Finding data in sorted collections
- Used in various algorithms as a subroutine

12 Common Interview Problems

1. Implement linear search.
2. Implement binary search (iterative and recursive).
3. Find the first and last occurrence of an element in a sorted array.
4. Search in a rotated sorted array.
5. Find a peak element in an array.
6. Find the minimum element in a rotated sorted array.
7. Search in a 2D matrix (sorted rows and columns).
8. Implement search in nearly sorted array.
9. Find the square root using binary search.
10. Find the position of an element in an infinite sorted array.

Rahul Yadav.

13. Sorting Algorithms in Data Structures

1 What is Sorting ?

- Sorting is the process of arranging elements in a specific order (ascending or descending).
- It makes searching, data processing and other operations more efficient.
- Sorting algorithms can be categorized as:
  - Comparison based (Bubble, Selection, Insertion, Merge, Quick, Heap)
  - Non-comparison based (Counting, Radix, Bucket)

2 Types of Sorting Algorithms

- Bubble Sort : Repeatedly swaps adjacent elements.
- Selection Sort : Selects the minimum/maximum element and places it at the correct position.
- Insertion Sort : Builds the sorted array one element at a time.
- Merge Sort : Divide and conquer approach.
- Quick Sort : Partition based divide and conquer.
- Heap Sort : Uses heap data structure (binary heap)
- Others : Counting Sort, Radix Sort, Bucket Sort, etc.

3 Bubble Sort

- Compare adjacent elements and swap if they are in wrong order.
- Repeat until the array is sorted.
- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Java Code (Bubble Sort)

```
public void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Example

Input : [5, 1, 4, 2, 8]
Pass 1: [1, 4, 2, 5, 8]
Pass 2: [1, 2, 4, 5, 8]
Pass 3: [1, 2, 4, 5, 8]

4 Selection Sort

- Find the minimum element from unsorted part and place it at the beginning.
- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Java Code (Selection Sort)

```
public void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx])
                minIdx = j;
        }
        int temp = arr[i];
        arr[i] = arr[minIdx];
    }   arr[minIdx] = temp;
}
```

Example

Input : [64, 25, 12, 22, 11]
Pass 1: [11, 25, 12, 22, 64]
Pass 2: [11, 12, 25, 22, 64]
Pass 3: [11, 12, 22, 25, 64]
Pass 4: [11, 12, 22, 25, 64]

5 Insertion Sort

- Pick an element and insert it into its correct position in the sorted part.
- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Java Code (Insertion Sort)

```
public void insertionSort(int[] arr) {
    int n = arr.length;
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Example

Input : [5, 2, 4, 6, 1]
Pass 1: [2, 5, 4, 6, 1]
Pass 2: [2, 4, 5, 6, 1]
Pass 3: [2, 4, 5, 6, 1]
Pass 4: [1, 2, 4, 5, 6]

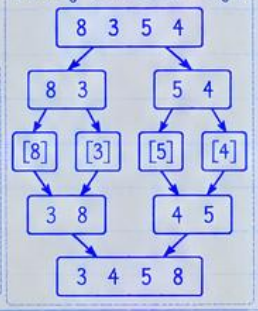
6 Merge Sort

- Divide the array into halves, sort them recursively and merge the sorted halves.
- Time Complexity: $O(n \log n)$
- Space Complexity: $O(n)$

Java Code (Merge Sort)

```
public void mergeSort(int[] arr, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);
    merge(arr, l, m, r);
}
```

Working (Divide and Merge)



7 Quick Sort

- Choose a pivot, partition the array and recursively sort the subarrays.
- Time Complexity: $O(n \log n)$ average, $O(n^2)$ worst case
- Space Complexity: $O(\log n)$

Java Code (Quick Sort)

```
public void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Partition Example

Input : [10, 7, 8, 9, 1, 5]
Pivot : 5
After Partition:
[1] [5] [10, 7, 8, 9]
< 5 5 > 5

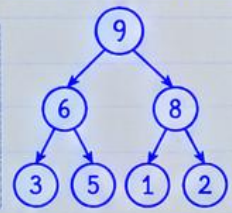
8 Heap Sort

- Build a Max Heap (or Min Heap) and repeatedly extract the root element.
- Time Complexity: $O(n \log n)$
- Space Complexity: $O(1)$

Java Code (Heap Sort)

```
public void heapSort(int[] arr) {
    int n = arr.length;
    buildMaxHeap(arr, n);
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}
```

Max Heap Example



9 Comparison of Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stable	In-place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes

10 Other Sorting Algorithms

- Counting Sort : $O(n + k)$, works for integers in a small range.
- Radix Sort : Sorts numbers digit by digit.
- Bucket Sort : Distributes elements into buckets.
- Tim Sort : Hybrid sorting (used in Java, Python).

11 Common Interview Problems

1. Implement bubble sort / selection sort / insertion sort.
2. Implement merge sort / quick sort.
3. Find the kth smallest element (using quick sort / heap).
4. Sort a nearly sorted array.
5. Sort an array of 0s, 1s and 2s (Dutch National Flag).
6. Find minimum number of swaps to sort an array.

Rahul Yadav.

14. Greedy Algorithms in Data Structures

1 What is Greedy Algorithm ?

- A greedy algorithm builds the solution step by step by always choosing the locally optimal choice at each step, hoping to find the global optimum.
- It makes the best choice at the current step without worrying about the future.
- Works for problems that have:
  - Greedy choice property
  - Optimal substructure
- Does not always give the optimal solution for all problems.
- Commonly used in optimization problems.

2 Characteristics and Properties

- Greedy Choice Property : A locally optimal choice leads to a globally optimal solution.
- Optimal Substructure : Optimal solution can be constructed from optimal solutions of subproblems.
- Usually easier and faster than dynamic programming.
- Does not consider all possible solutions (unlike DP).
- Time complexity is usually O(n) or O(n log n).

3 Common Greedy Algorithm Problems

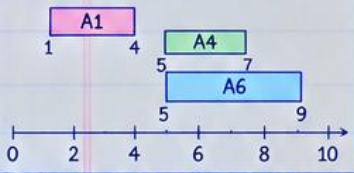
- Activity Selection Problem
- Fractional Knapsack Problem
- Huffman Coding
- Minimum Spanning Tree (Kruskal's, Prim's)
- Dijkstra's Shortest Path (non-negative weights)
- Job Sequencing with Deadlines
- Coin Change Problem (for specific coin denominations)

4 Activity Selection Problem

- Given n activities with start and end times, select the maximum number of non-overlapping activities.
- Greedy Strategy: Always choose the activity with the earliest finish time.

Example Input

Activity	Start	End
A1	1	4
A2	3	5
A3	0	6
A4	5	7
A5	3	9
A6	5	9



Java Code (Activity Selection)

```
class Activity {
    int start, end;
    Activity(int s, int e) {
        start = s; end = e;
    }
}
static void activitySelection(Activity[] arr) {
    Arrays.sort(arr, (a, b) -> a.end - b.end);
    System.out.println("Selected Activities:");
    int lastEnd = -1;
    for (Activity act : arr) {
        if (act.start >= lastEnd) {
            System.out.println("(" + act.start
                + ", " + act.end + ")");
            lastEnd = act.end;
        }
    }
}
```

5 Fractional Knapsack Problem

- Given n items with weight and value, and a knapsack with capacity W, find the maximum possible value.
- We can take fractional part of items.
- Greedy Strategy: Choose item with highest value-to-weight ratio first.

Item	Weight	Value	Value/Weight
1	10	60	6.0
2	20	100	5.0
3	30	120	4.0

Knapsack Capacity = 50
Take item 1 (10), item 2 (20),
20/30 of item 3.
Maximum Value = 60 + 100 + 80 = 240

Java Code (Fractional Knapsack)

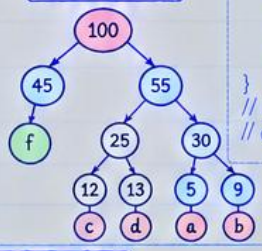
```
static double fractionalKnapsack(int W, int[] wt, int[] val) {
    int n = wt.length;
    double[][] items = new double[n][3];
    for (int i = 0; i < n; i++) {
        items[i][0] = i;
        items[i][1] = wt[i];
        items[i][2] = val[i];
    }
    Arrays.sort(items, (a, b) -> Double.compare(
        (b[2]/b[1]), (a[2]/a[1])));
    double maxValue = 0.0; int capacity = W;
    for (int i = 0; i < n; i++) {
        int w = (int) items[i][1];
        int v = (int) items[i][2];
        if (capacity >= w) {
            maxValue += v; capacity -= w;
        } else {
            maxValue += ((double) v / w) * capacity;
            break;
        }
    }
    return maxValue;
}
```

6 Huffman Coding

- Given a set of characters with frequencies, build a binary tree to get the minimum length encoding.
- Greedy Strategy: Always merge two nodes with the smallest frequency.

Character	Frequency
a	5
b	9
c	12
d	13
e	16
f	45

Huffman Tree



Java Code (Huffman Coding)

```
class Node implements Comparable<Node> {
    char ch; int freq;
    Node left, right;
    Node(char ch, int freq) {
        this.ch = ch; this.freq = freq;
    }
    public int compareTo(Node n) {
        return this.freq - n.freq;
    }
}
// Use PriorityQueue<Node> to build tree
// and generate codes (standard implementation)
```

7 Other Greedy Algorithms

- Minimum Spanning Tree (MST) : Kruskal's and Prim's algorithm.
- Dijkstra's Shortest Path : For non-negative weights.
- Job Sequencing with Deadlines : Maximize profit.
- Coin Change (Greedy) : Works for specific denominations (e.g., 1, 5, 10, 20...).

8 Time and Space Complexity

Problem	Time Complexity	Space Complexity
Activity Selection	O(n log n)	O(1)
Fractional Knapssck	O(n log n)	O(1)
Huffman Coding	O(n log n)	O(n)
Kruskal's MST	O(E log E)	O(V)
Prim's MST	O(E log V)	O(V)
Dijkstra's	O((V + E) log V)	O(V)

9 Advantages and Disadvantages

Advantages

- ✔ Simple and easy to implement.
- ✔ Efficient (usually O(n) or O(n log n)).
- ✔ Works well for many optimization problems.
- ✔ Less memory usage compared to DP.

Disadvantages

- ✘ Does not always give the optimal solution.
- ✘ Works only for problems with greedy choice property and optimal substructure.
- ✘ Not suitable for all types of problems (e.g., 0/1 Knapsack).

10 Common Interview Problems

1. Activity selection problem.
2. Fractional knapsack problem.
3. Huffman coding.
4. Minimum spanning tree (Kruskal's / Prim's).
5. Dijkstra's shortest path.
6. Job sequencing with deadlines.
7. Coin change problem (greedy approach).
8. Find minimum number of platforms (greedy).
9. N meetings in one room.
10. Minimum cost to connect all ropes.

Rahul Yadav.

17. Bit Manipulation in Data Structures

1 What is Bit Manipulation ?

- Bit manipulation means performing operations directly on the binary representation of numbers.
- Each integer is represented using bits (0 and 1) in memory.
- Bit manipulation helps in writing efficient algorithms, solving complex problems, and optimizing space and time.
- It is widely used in competitive programming, system design, cryptography, networking, and low-level programming.

2 Binary Representation

- Computers store integers in binary form (base 2).
- Each bit can be 0 or 1.
- Example (8-bit representation) :

5 = 00000101

10 = 00001010

−5 (2's complement) :

→ Invert bits : 11111010

→ Add 1 : 11111011

3 Common Bitwise Operators

Operator	Name	Description	Example (5 and 3)
&	AND	1 if both bits are 1	5 & 3 = 1 (0101 & 0011)
|	OR	1 if at least one bit is 1	5 | 3 = 7 (0101 | 0011)
^	XOR	1 if bits are different	5 ^ 3 = 6 (0101 ^ 0011)
~	NOT	Inverts all bits	~5 = −6 (2's complement)
<<	Left Shift	Multiply by 2^n	5 << 1 = 10
>>	Right Shift	Divide by 2^n	5 >> 1 = 2

4 Bitwise AND Example (5 & 3)

```
5 =  0101
3 =  0011
-----------
5 & 3 = 0001 (1)
```

5 Bitwise OR Example (5 | 3)

```
5 =  0101
3 =  0011
-----------
5 | 3 = 0111 (7)
```

6 Bitwise XOR Example (5 ^ 3)

```
5 =  0101
3 =  0011
-----------
5 ^ 3 = 0110 (6)
```

7 Left and Right Shift

a = 5 (0101)

- Left Shift (a << 1): 1010 (10) → a * 2
- Right Shift (a >> 1): 0010 (2) → a / 2

8 Useful Bit Manipulation Tricks

1. Check if a number is even or odd : n & 1
2. Multiply by 2 : n << 1
3. Divide by 2 : n >> 1
4. Check if kth bit is set : (n & (1 << k)) != 0
5. Set kth bit : n | (1 << k)
6. Clear kth bit : n & (~(1 << k))
7. Toggle kth bit : n ^ (1 << k)
8. Count set bits : Integer.bitCount(n)
9. Check if number is power of 2 : (n > 0) && (n & (n-1)) == 0
10. Find rightmost set bit : n & -n
11. Remove rightmost set bit : n & (n - 1)
12. Swap two numbers : a = a ^ b ; b = a ^ b ; a = a ^ b;
13. Find unique number (all others appear twice) using XOR
14. Check if two numbers have opposite signs : (a ^ b) < 0

9 Check if a Number is Power of 2

- A number is power of 2 if it has only one set bit.
- Condition : (n > 0) && (n & (n − 1)) == 0

Java Code

```
public static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

Example

8 (1000) → true
16 (10000) → true
10 (1010) → false

10 Count Number of Set Bits (Hamming Weight)

- Use Brian Kernighan's algorithm : n & (n − 1)

Java Code

```
public static int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        n = n & (n - 1);
        count++;
    }
    return count;
}
```

Example

n = 13 (1101)
Set bits = 3
Steps:
1101 → 1100 → 1000 → 0000
Count = 3

11 Find Single Number (All elements twice except one)

- Use XOR. XOR of a number with itself is 0 and XOR with 0 is the number itself.
- Works because a ^ a = 0 and a ^ 0 = a.
- Time Complexity: O(n)
- Space Complexity: O(1)

Java Code

```
public static int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) {
        result ^= n;
    }
    return result;
}
```

Example

Input : [4, 1, 2, 1, 2]
Output : 4

12 Find Two Unique Numbers (All elements twice except two)

- Use XOR and rightmost set bit to separate two numbers.
- Time Complexity: O(n)
- Space Complexity: O(1)

Java Code

```
public static int[] twoUniqueNumbers(int[] nums) {
    int xr = 0;
    for (int n : nums) xr ^= n;
    int rightSetBit = xr & -xr;
    int x = 0, y = 0;
    for (int n : nums) {
        if ((n & rightSetBit) != 0) x ^= n;
        else y ^= n;
    }
    return new int[]{x, y};
}
```

Example

Input : [1, 2, 3, 2, 1, 4]
Output : [3, 4]

13 Common Interview Problems

1. Find single number (others appear twice).
2. Find two unique numbers.
3. Check if a number is power of 2.
4. Count number of set bits.
5. Reverse bits of a number.
6. Find missing number (1 to n).
7. Find the number with odd frequency.
8. Subsets using bit masking.
9. Gray code.
10. Bitwise AND of numbers in a range.

14 Time and Space Complexity

Problem	Time Complexity	Space Complexity
Check even/odd	O(1)	O(1)
Set/Clear/Toggle bit	O(1)	O(1)
Count set bits	O(1)	O(1)
Single number	O(n)	O(1)
Two unique numbers	O(n)	O(1)
Power of 2 check	O(1)	O(1)
Reverse bits	O(1)	O(1)

15 Applications

- Competitive programming
- System level programming (operating systems, compilers)
- Cryptography and security
- Networking (IP, subnet masks)
- Memory optimization
- Game development (bitboards)
- Database and indexing
- Compression algorithms
- Embedded systems

Rahul Yadav.

16. Sliding Window and Two Pointers in Data Structures

1 What is Sliding Window ?

- Sliding Window is a technique used to solve problems on arrays or strings where we need to consider a subarray or substring of a fixed or variable size.
- Instead of using nested loops, we maintain a window (range) and slide it across the data structure while updating the result efficiently.
- It reduces the time complexity from $O(n^2)$ to $O(n)$ for many problems.

2 Types of Sliding Window

- **Fixed Size Window**
  Window size is constant.
- **Variable Size Window**
  Window size changes based on a condition (expand/shrink).

3 When to Use ?

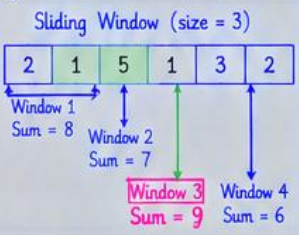
- Problems involving subarray or substring.
- Find maximum/minimum/sum in a window.
- Count elements based on a condition.
- Longest/shortest subarray or substring.
- Optimize problems from $O(n^2)$ to $O(n)$.

4 Fixed Size Sliding Window Example

- Find the maximum sum of subarray of size k.

Example:

Array : [2, 1, 5, 1, 3, 2]
k = 3
Output : 9 (subarray [5, 1, 3])



Java Code (Fixed Size Window)

```
public static int maxSumSubarray(int[] arr, int k) {
    int windowSum = 0, maxSum = 0;
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }
    maxSum = windowSum;
    for (int i = k; i < arr.length; i++) {
        windowSum += arr[i] - arr[i - k];
        maxSum = Math.max(maxSum, windowSum);
    return maxSum;
}
```

Time and Space Complexity

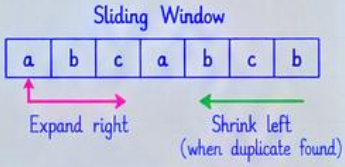
Time : O(n)
Space : O(1)

5 Variable Size Sliding Window Example

- Find the length of the longest substring without repeating characters.

Example:

String : "abcabcbb"
Output : 3 ("abc")



Java Code (Longest Substring Without Repeating Characters)

```
public static int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        while (set.contains(s.charAt(right))) {
            set.remove(s.charAt(left));
            left++;
        }
        set.add(s.charAt(right));
        maxLen = Math.max(maxLen, right-left+1);
    }
    return maxLen;
}
```

Time Complexity : O(n)
Space Complexity : O(min(n, m))
m = size of character set

6 Two Pointers Technique

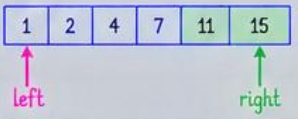
- Use two pointers to solve problems efficiently.
- Commonly used in sorted arrays, linked lists and strings.
- Can be used to find pairs, triplets, or subarrays.
- Usually reduces time complexity from $O(n^2)$ to $O(n)$.

7 Two Pointers Example (Sorted Array)

Find a pair with given sum in a sorted array.

Example:

Array : [1, 2, 4, 7, 11, 15]
Target : 15
Output : Pair (4, 11)



Java Code (Two Sum in Sorted Array)

```
public static int[] twoSumSorted(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) {
            return new int[]{arr[left], arr[right]};
        } else if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
    return new int[]{}; // no pair found
}
```

Time Complexity : O(n)
Space Complexity : O(1)

8 Two Pointers Example (Container With Most Water)

- Given an array of heights, find the maximum area of water that can be contained.

Example:

Heights : [1, 8, 6, 2, 5, 4, 8, 3, 7]
Output : 49



Java Code (Container With Most Water)

```
public static int maxArea(int[] height) {
    int left = 0, right = height.length - 1;
    int maxArea = 0;
    while (left < right) {
        int h = Math.min(height[left], height[right]);
        int area = h * (right - left);
        maxArea = Math.max(maxArea, area);
        if (height[left] < height[right]) left++;
        else right--;
    }return maxArea;
}
```

Time Complexity : O(n)
Space Complexity : O(1)

9 Common Problems Using Sliding Window / Two Pointers

1. Maximum sum subarray of size k
2. Longest substring without repeating characters
3. Smallest substring containing all characters
4. Count subarrays with sum equal to k
5. Minimum size subarray sum ≥ target
6. Find all anagrams in a string
7. Longest subarray with at most K distinct elements
8. Two sum in sorted array
9. Three sum (sort + two pointers)
10. Container with most water
11. Trapping rain water
12. Remove duplicates from sorted array
13. Pair with given difference
14. Move zeroes
15. Dutch national flag problem

10 Advantages and Disadvantages

Advantages	Disadvantages
✓ Reduces time complexity (significantly).	✗ Not applicable to all problems.
✓ Efficient for subarray/substring problems.	✗ Careful handling required for edge cases.
✓ Simple and easy to implement in many cases.	✗ May not work directly for non-array data structures.
✓ Works well with sorted arrays (Two Pointers).	✗ For some problems, additional data structures (HashMap, Set) are needed.

Rahul Yadav.

15. Dynamic Programming in Data Structures

1 What is Dynamic Programming (DP) ?

- Dynamic Programming is a technique to solve complex problems by breaking them into smaller overlapping subproblems and storing their results to avoid redundant computations.
- It works for problems having:
  - Optimal Substructure : Optimal solution can be constructed from optimal solutions of subproblems.
  - Overlapping Subproblems : Same subproblems are solved multiple times.
  - It can be implemented using Memoization (Top-Down) or Tabulation (Bottom-Up).

2 Key Characteristics

- Overlapping subproblems
- Optimal substructure
- Stores results of subproblems (using DP table or recursion + memo)
- Reduces time complexity from exponential to polynomial
- Used in optimization problems (maximization or minimization)

3 Approaches in DP

1. Memoization (Top-Down)	2. Tabulation (Bottom-Up)
• Use recursion	• Use iterative approach
• Store results in a cache (array/map)	• Build dp table from smaller to larger subproblems
• Solve subproblems only once	• Generally more space efficient
• Generally easier to write	• Avoids recursion stack overflow

4 Fibonacci Series using DP

- Find the nth Fibonacci number.
  Formula: $F(n) = F(n-1) + F(n-2)$

Memoization (Top-Down)

```
public static int fibMemo(int n, int[] dp) {
    if (n <= 1) return n;
    if (dp[n] != -1) return dp[n];
    dp[n] = fibMemo(n - 1, dp) + fibMemo(n - 2, dp);
    return dp[n];
}
```

Tabulation (Bottom-Up)

```
public static int fibTab(int n) {
    if (n <= 1) return n;
    int[] dp = new int[n + 1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Example (n = 6)

i	F(i)
0	0
1	1
2	1
3	2
4	3
5	5
6	8

Time and Space Complexity

Approach	Time	Space
Naive Recursion	$O(2^n)$	$O(n)$
Memoization	$O(n)$	$O(n)$
Tabulation	$O(n)$	$O(n)$
Space Optimized	$O(n)$	$O(1)$

5 0/1 Knapsack Problem

- Given n items with weight and value, and a knapsack with capacity W, find the maximum value that can be obtained by selecting items (each item can be taken once).
- Recurrence:
  `dp[i][w] = max(dp[i - 1][w], val[i - 1] + dp[i - 1][w - wt[i - 1]])`

Example

Weights: [1, 3, 4, 5]
Values : [1, 4, 5, 7]
Capacity (W) = 7

Tabulation (Bottom-Up)

```
public static int knapsack(int[] wt, int[] val, int W) {
    int n = wt.length;
    int[][] dp = new int[n + 1][W + 1];
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (st[i - 1] <= w)
                dp[i][w] = Math.max(dp[i - 1][w],
                    val[i - 1] + dp[i - 1][w - wt[i - 1]]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }
    return dp[n][W];
}
```

DP Table (dp[i][w])

w →	0	1	2	3	4	5	6	7
i ↓	.	.	.	.	.	.	.	.
0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1
2	0	1	1	4	4	5	5	5
3	0	1	1	4	5	6	6	9
4	0	1	1	4	5	7	8	9

Maximum value = 9

6 Longest Common Subsequence (LCS)

- Given two strings, find the length of their longest common subsequence.

```
Recurrence:
    if (s1[i - 1] == s2[j - 1])
        dp[i][j] = 1 + dp[i - 1][j - 1]
    else
        dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
```

Tabulation (Bottom-Up)

```
public static int lcs(String s1, String s2) {
    int n = s1.length(), m = s2.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 1; i <= n; i++) {
        for (int j = 2; j <= m; j++) {
            if (s1.charAt(i - 1) == s2.charAt(j - 1))
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[n][m];
}
```

Example: s1 = "AGGTAB" , s2 = "GXTXAYB"
LCS length = 4 ("GTAB")

7 Other Common DP Problems

- Climbing Stairs (Fibonacci variant)
- Minimum Cost Climbing Stairs
- Coin Change Problem (Minimum coins & Number of ways)
- Edit Distance (Levenshtein Distance)
- Matrix Chain Multiplication (MCM)
- Partition Equal Subset Sum
- Subset Sum
- Rod Cutting Problem
- Longest Increasing Subsequence (LIS)
- Palindrome Partitioning (Minimum cuts)
- Maximum Sum Subarray (DP variant)
- Stock Buy and Sell (multiple variants)
- Burst Balloons
- Unique Paths (with/without obstacles)

8 Time and Space Complexity (Common DP Problems)

Problem	Time Complexity	Space Complexity
Fibonacci	$O(n)$	$O(n)$ / $O(1)$
0/1 Knapsack	$O(nW)$	$O(nW)$
LCS	$O(nm)$	$O(nm)$
Edit Distance	$O(nm)$	$O(nm)$
Matrix Chain Multiplication	$O(n^3)$	$O(n^2)$
Coin Change	$O(nW)$	$O(nW)$
Longest Increasing Subsequence	$O(n^2)$	$O(n)$

When to use DP ?

- When the problem has overlapping subproblems.
- When the problem has optimal substructure.
- When brute force leads to exponential time complexity.
- Commonly used in optimization and counting problems.

9 Advantages and Disadvantages

Advantages

- ✓ Reduces time complexity significantly.
- ✓ Efficient for optimization problems.
- ✓ Avoids redundant computations.
- ✓ Can be implemented using recursion or iteration.

Disadvantages

- ✗ May require extra space for DP table.
- ✗ Not applicable to all problems.
- ✗ Can be complex to identify the state and recurrence relation.

10 Common Interview Problems

1. Fibonacci number.
2. 0/1 Knapsack problem.
3. Longest Common Subsequence.
4. Longest Increasing Subsequence.
5. Edit distance.
6. Coin change (min coins / number of ways).
7. Matrix chain multiplication.
8. Partition equal subset sum.
9. Stock buy and sell (I, II, II, IV).
10. Minimum cost climbing stairs.

Rahul Yadav.

18. Tries (Prefix Trees) in Data Structures

1 What is Trie ?

- Trie (pronounced "try") is a tree-like data structure used to store a collection of strings (usually words).
- Each node represents a single character.
- Commonly used for prefix-based searching.
- Efficient for searching, inserting and deleting strings.
- It is also called Prefix Tree.
- Time complexity depends on length of the string, not on the number of strings stored.

2 Key Characteristics

- Stores strings in a tree structure.
- Each edge represents a character.
- Path from root to a node represents a prefix.
- Each node may have multiple children (alphabet size).
- A node can mark the end of a word (isEndOfWord = true).
- Useful for autocomplete, dictionary, spell check, etc.
- Space complexity is usually O(n * m) where n = number of words, m = average length of word.

3 Applications

- Autocomplete / Search suggestions
- Dictionary and spell check
- IP routing (longest prefix match)
- String matching (prefix search)
- Phone directory
- Boggle game (word search)
- Data compression
- Finding words with given prefix
- Counting words with prefix
- Solving problems in competitive programming

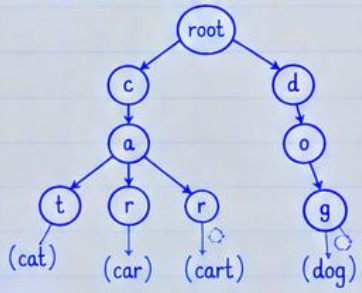
4 Trie Node Structure

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEndOfWord = false;
}
```

- We use an array of size 26 for lowercase English alphabets (a-z).
- children[i] represents the next character.
- isEndOfWord is true if the node represents end of a word.

5 Example Trie for Words

Insert words : ["cat", "car", "cart", "dog"]



6 Operations in Trie

- Insert a word
- Search a word
- Check if a word starts with a prefix
- Delete a word
- Count words with given prefix
- Find all words with a given prefix
- Longest common prefix
- Autocomplete suggestions

7 Insert Operation (Java Code)

```
class Trie {
    TrieNode root = new TrieNode();

    public void insert(String word) {
        TrieNode node = root;
        for (char ch : word.toCharArray()) {
            int index = ch - 'a';
            if (node.children[index] == null) {
                node.children[index] = new TrieNode();
            }
            node = node.children[index];
        }
        node.isEndOfWord = true;
    }
}
```

8 Search Operation (Java Code)

```
public boolean search(String word) {
    TrieNode node = root;
    for (char ch : word.toCharArray()) {
        int index = ch - 'a';
        if (node.children[index] == null) {
            return false;
        }
        node = node.children[index];
    }
    return node.isEndOfWord;
}
```

Example:

```
search("car")  → true
search("cat")  → true
search("can")  → false
```

9 Starts With (Prefix Search)

```
public boolean startsWith(String prefix) {
    TrieNode node = root;
    for (char ch : prefix.toCharArray()) {
        int index = ch - 'a';
        if (node.children[index] == null) {
            return false;
        }
        node = node.children[index];
    }
    return true;
}
```

Example:

```
startsWith("ca")  → true
startsWith("do")  → true
startsWith("z")   → false
```

10 Delete Operation (Java Code)

```
public boolean delete(String word) {
    return deleteHelper(root, word, 0);
}

private boolean deleteHelper(TrieNode node, String word, int depth) {
    if (node == null) return false;
    if (depth == word.length()) {
        if (!node.isEndOfWord) return false;
        node.isEndOfWord = false;
        return isEmpty(node);
    }
    int index = word.charAt(depth) - 'a';
    if (!deleteHelper(node.children[index], word, depth + 1))
        return false;
    if (isEmpty(node.children[index])) {
        node.children[index] = null;
        return !node.isEndOfWord && isEmpty(node);
    }
    return false;
}

private boolean isEmpty(TrieNode node) {
    for (TrieNode child : node.children) {
        if (child != null) return false;
    }
    return true;
}
```

11 Find All Words with Given Prefix

```
public List<String> getWordsWithPrefix(String prefix) {
    List<String> result = new ArrayList<>();
    TrieNode node = root;
    for (char ch : prefix.toCharArray()) {
        int index = ch - 'a';
        if (node.children[index] == null) {
            return result;
        }
        node = node.children[index];
    }
    dfs(node, new StringBuilder(prefix), result);
    return result;
}

private void dfs(TrieNode node, StringBuilder path,
                 List<String> result) {
    if (node.isEndOfWord) {
        result.add(path.toString());
    }
    for (int i = 0; i < 26; i++) {
        if (node.children[i] != null) {
            path.append((char)('a' + i));
            dfs(node.children[i], path, result);
            path.deleteCharAt(path.length() - 1);
        }
    }
}
```

12 Time and Space Complexity

Operation	Time Complexity	Space Complexity
Insert	O(m)	-
Search	O(m)	-
Starts With	O(m)	-
Delete	O(m)	-
Find all words	O(m + k)	-
Space	-	O(n * m)

m = length of the string
n = number of words
k = total length of all words with given prefix

13 Common Interview Problems

1. Implement a Trie (Insert, Search, Delete).
2. Find if a word exists in a Trie.
3. Find all words with a given prefix.
4. Longest common prefix of a set of strings.
5. Word search (Boggle using Trie).
6. Replace words (LeetCode 648).
7. Design autocomplete system.
8. Count distinct substrings using Trie.
9. Maximum XOR pair in an array (using Trie).
10. Implement a dictionary with suggestions.

Rahul Yadav.

19. Disjoint Set Union (DSU) in Data Structures

1 What is Disjoint Set Union ?

- Disjoint Set Union (DSU) or Union-Find is a data structure that keeps track of a set of elements partitioned into disjoint (non-overlapping) sets.
- It efficiently supports two operations:
  - Find: Find the representative (parent) of a set.
  - Union: Merge two sets.
- It is mainly used to determine whether two elements belong to the same set.
- It is widely used in graph algorithms, connected components, cycle detection, Kruskal's Minimum Spanning Tree, etc.

2 Key Concepts

- Each set has a representative element called the parent or leader (root).
- Initially, each element is in its own set (parent[i] = i).
- Union operation merges two sets.
- Find operation returns the root/parent of a set.
- Path Compression: While finding the parent, make each visited node point directly to the root to reduce tree height.
- Union by Rank/Size: Always attach the smaller tree under the larger tree to keep the tree shallow.

3 Advantages

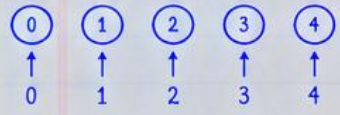
- Very efficient operations (almost O(1)).
- Useful for dynamic connectivity problems.
- Simple to implement.
- Optimized using path compression and union by rank/size.

Applications

- Kruskal's Minimum Spanning Tree (MST)
- Detect cycle in an undirected graph
- Find connected components in a graph
- Dynamic connectivity (network problems)
- Image processing (connected pixels)
- Accounts merge problem
- Friend circles / social network

4 Basic Example

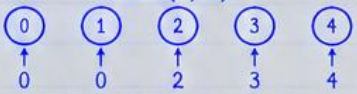
Elements: {0, 1, 2, 3, 4}
Initially each element is in its own set.



parent[] = [0, 1, 2, 3, 4]
rank[] = [0, 0, 0, 0, 0]

5 Union Operation Example

Union(0, 1)



parent[] = [0, 0, 2, 3, 4]
rank[] = [1, 0, 0, 0, 0]

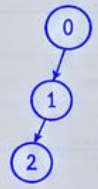
Union(1, 2)



parent[] = [0, 0, 2, 3, 4]
rank[] = [1, 0, 0, 0, 0]

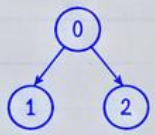
6 Find Operation with Path Compression

find(2) before path compression:



parent[2] = 1
parent[1] = 0
Root = 0

After path compression:



parent[2] = 0
parent[1] = 0

Now 2 directly points to root.

7 DSU Implementation in Java (Path Compression + Union by Rank)

```java
class DisjointSetUnion {
    private int[] parent;
    private int[] rank;
    public DisjointSetUnion(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            rank[i] = 0;
        }
    }
    // Find with path compression
    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }
        return parent[x];
    }
    // Union by rank
    public void union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);
        if (rootX == rootY) return;
        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else if (rank[rootX] > rank[rootY]) {
            parent[rootY] = rootX;
        } else {
            parent[rootY] = rootX;
            rank[rootX] ++;
        }
    }
    // Check if two elements are in same set
    public boolean isConnected(int x, int y) {
        return find(x) == find(y);
    }
}
```

8 Java Code Example (Usage)

```java
public class Main {
    public static void main(String[] args) {
        DisjointSetUnion dsu = new DisjointSetUnion(5);
        dsu.union(0, 1);
        dsu.union(1, 2);
        System.out.println(dsu.isConnected(0, 2)); // true
        System.out.println(dsu.isConnected(0, 3)); // false
        dsu.union(3, 4);
        System.out.println(dsu.find(4)); // 3
        dsu.union(2, 4);
        System.out.println(dsu.isConnected(0, 4)); // true
    }
}
```

Output

```
true
false
3
true
```

9 Time and Space Complexity

Operation	Time Complexity	Space Complexity
Find (with path compression)	O(α(n))	O(1)
Union (by rank)	O(α(n))	O(1)
Initialize	O(n)	O(n)

- α(n) is the inverse Ackermann function, which grows extremely slow and can be considered as constant for all practical purposes.
- So both Find and Union operations are almost O(1).

10 Common Interview Problems

1. Number of Connected Components in an Undirected Graph
2. Detect Cycle in an Undirected Graph
3. Kruskal's Minimum Spanning Tree
4. Accounts Merge (LeetCode 721)
5. Number of Provinces (LeetCode 547)
6. Redundant Connection (LeetCode 684)
7. Similar String Groups (LeetCode 839)
8. Most Stones Removed with Same Row or Column (LeetCode 947)
9. Satisfiability of Equality Equations (LeetCode 990)
10. Dynamic Connectivity in a Grid

11 Summary

- DSU maintains a collection of disjoint sets.
- Use path compression to flatten the tree during find.
- Use union by rank/size to keep the tree shallow.
- Find and Union operations are almost O(1).
- Widely used in graph problems and competitive programming.

12 Key Points to Remember

- Always use both path compression and union by rank for optimal performance.
- Parent of a node represents the set leader (root).
- If find(x) == find(y), then x and y are in the same set.
- Union merges the sets if they are different.
- DSU is applicable for undirected graphs and connectivity problems.

Rahul Yadav.

20. Advanced DSA - Master Cheat Sheet

1 Important Algorithmic Patterns

- Sliding Window (subarray, substring)
- Two Pointers (pairs, triplets, sorted arrays)
- Fast and Slow Pointers (linked list, cycles)
- Greedy (local optimal to global optimal)
- Dynamic Programming (overlapping subproblems)
- Divide and Conquer (split and combine)
- Backtracking (try all possibilities)
- Bit Manipulation (optimize using bits)
- Graph Traversal (BFS, DFS)
- Disjoint Set Union (merge components)
- Trie (prefix based problems)
- Heap / Priority Queue (top K, scheduling)

2 Problem Solving Steps

1. Understand the problem statement.
2. Identify input, output and constraints.
3. Think of brute force solution first.
4. Identify the correct algorithmic pattern.
5. Optimize the solution (time and space).
6. Handle edge cases (empty input, single element).
7. Analyze time and space complexity.
8. Dry run with examples.
9. Write clean and modular code.
10. Test with different test cases.

3 Time and Space Complexity Guide

Operation / Data Structure	Time Complexity	Space Complexity
Array Access	O(1)	O(1)
Array Search	O(n)	O(1)
Array Insert/Delete	O(n)	O(1)
HashMap (put/get)	O(1)	O(n)
HashMap (worst)	O(n)	O(n)
ArrayList (get)	O(1)	O(n)
LinkedList (insert/delete)	O(1)	O(n)
Stack (push/pop)	O(1)	O(n)
Queue (offer/poll)	O(1)	O(n)
PriorityQueue (offer/poll)	O(log n)	O(n)
Tree (BST search)	O(log n)	O(n)
Trie (insert/search)	O(m)	O(n * m)
Union-Find (amortized)	O(α(n))	O(n)

4 Common Graph Algorithms

- BFS - Shortest path in unweighted graph.
- DFS - Explore all possibilities.
- Dijkstra - Shortest path in weighted graph.
- Bellman-Ford - Handles negative weights.
- Floyd-Warshall - All pairs shortest path.
- Kruskal's - Minimum Spanning Tree.
- Prim's - Minimum Spanning Tree.
- Topological Sort - For Directed Acyclic Graph.
- Cycle Detection - Using DFS / Union-Find.

5 Typical DP Patterns

- 1D DP (Fibonacci, Climbing Stairs)
- 2D DP (Knapasck, LCS, Edit Distance)
- DP on Strings
- DP on Subsequences
- DP on Subsets (Bitmask DP)
- DP on Tree (Tree DP)
- Memoization (Top Down)
- Tabulation (Bottom Up)
- State Transition and Base Cases

6 Sorting Algorithm Complexities

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No

7 Binary Search Variants

- Find element in sorted array.
- Find first and last occurrence.
- Search in rotated sorted array.
- Find minimum in rotated array.
- Search in 2D matrix.
- Binary search on answer (optimization).
- Find peak element.
- Split array / minimize maximum.

8 Bit Manipulation Tricks

- Check even/odd : `n & 1`
- Multiply by 2 : `n << 1`
- Divide by 2 : `n >> 1`
- Set kth bit : `n | (1 << k)`
- Clear kth bit : `n & ~(1 << k)`
- Toggle kth bit : `n ^ (1 << k)`
- Count set bits : `Integer.bitCount(n)`
- Check power of 2 : `(n > 0) && (n & (n-1)) == 0`
- Swap two numbers : `a ^= b; b ^= a; a ^= b;`
- Find unique number : XOR of all elements

9 Recursion vs Iteration

Recursion	Iteration
• Function calls itself.	• Uses loops (for, while).
• Uses call stack.	• Generally uses less space.
• More readable for some problems.	• More efficient in most cases.
• May have higher space complexity.	• Better control over memory.
• Useful in trees, graphs, backtracking.	• Useful in simple linear problems.

10 Backtracking Pattern (Template)

```
public void backtrack(...) {
    if (baseCondition) {
        // store result
        return;
    }
    for (each choice) {
        // make choice
        backtrack(...);
        // undo choice (backtrack)
    }
}
```

11 Top K Problems (Using Heap)

- Kth largest/smallest element.
- Top K frequent elements.
- K closest points to origin.
- Merge K sorted lists.
- Find median from data stream.
- Kth largest in a stream.

12 Common Data Structure Use Cases

Data Structure	Use Cases
Array	Random access, simple storage
HashMap	Fast lookups, counting, frequency
HashSet	Unique elements, membership check
LinkedList	Frequent insert/delete
Stack	Undo, expression evaluation, DFS
Queue	BFS, scheduling, level order traversal
PriorityQueue	Top K, smallest/largest element
Tree (BST)	Sorted data, range queries
Trie	Prefix search, autocomplete
Union-Find	Connected components, cycle detection
Graph	Network, path finding, dependencies

13 Common Interview Problem Types

1. Array and String manipulation
2. Two Pointers / Sliding Window
3. Sorting and Searching
4. Matrix problems
5. Linked List (reverse, cycle, merge)
6. Tree (traversals, LCA, serialization)
7. Graph (BFS, DFS, shortest path)
8. Dynamic Programming
9. Greedy Algorithms
10. Backtracking (permutations, combinations)
11. Heap / Priority Queue
12. Trie (autocomplete, dictionary)
13. Bit Manipulation
14. Disjoint Set Union (Union-Find)
15. Design Problems (LRU Cache, Twitter, etc.)

14 Tips for Interviews

- Practice coding regularly (2 problems per day).
- Focus on problem solving patterns.
- Write clean and readable code.
- Analyze time and space complexity.
- Discuss brute force to optimal approach.
- Handle edge cases and invalid inputs.
- Draw diagrams and dry run examples.
- Be comfortable with common data structures.
- Practice previous year questions (LeetCode).
- Communicate your thought process clearly.

15 Final Revision Checklist

- ☐ Arrays, Strings, Hashing
- ☐ Two Pointers, Sliding Window
- ☐ Binary Search and Variations
- ☐ Sorting Algorithms
- ☐ Linked List
- ☐ Stacks and Queues
- ☐ Trees (Binary Tree, BST)
- ☐ Graphs (BFS, DFS, Dijkstra, MST)
- ☐ Dynamic Programming
- ☐ Greedy Algorithms
- ☐ Backtracking
- ☐ Heaps and Priority Queue
- ☐ Tries
- ☐ Bit Manipulation
- ☐ Disjoint Set Union
- ☐ Practice 100+ problems

Rahul Yadav.